



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – ARQUITETURA E ORGANIZAÇÃO DE
PROCESSADORES
PROF. THIAGO FELSKI PEREIRA

GUSTAVO HENRIQUE STAHL MÜLLER
PAULO HENRIQUE ROHLING

PROGRAMAÇÃO EM LINGUAGEM DE MONTAGEM

1. INTRODUÇÃO

O objetivo deste trabalho é apresentar o desenvolvimento e funcionamento de dois programas simples, criados no Assembly da arquitetura MIPS.

2. PRIMEIRO PROGRAMA

“Crie um programa que leia um número inteiro e uma operação via console e, após a leitura do número, execute a operação desejada:

0 – Encerra o programa;

1 – Calcula e imprime o resto da divisão inteira por 2 sem usar o operador mod;

2 – Calcula e imprime o resultado da divisão inteira por 2 do número digitado sem usar o operador div.”

Código fonte em linguagem de alto nível (C++):

```
#include <iostream>

using namespace std;

int main() {
    // Pegar o valor da variável
    int val = 0;
    cout << "Entre com um valor inteiro (maior do que zero)." << endl;

    while(val < 1) {
        cin >> val;
        if(val > 0) {
            break;
        }
        cout << "Valor invalido." << endl;
    }
    cout << endl;

    // Menu de operações
    cout << "Digite uma operacao, sendo que:" << endl;
    cout << "0 = encerra o programa;" << endl;
    cout << "1 = informa o resto da divisao por 2;" << endl;
    cout << "2 = informa o resultado da divisao por 2." << endl;

    // Pegar a operação desejada
    int operation = -1;
    while(operation < 0 || operation > 2) {
        cin >> operation;
        if(operation > -1 && operation < 3) {
            break;
        }
        cout << "Operacao invalida" << endl;
    }

    cout << endl;

    // Terminar o programa
    if(operation == 0) {
        return 0;
    }
}
```

```

// Resto da divisão por 2
if(operation == 1) {
    int remainder;
    int result = val/2;
    int subtractor = result * 2;

    remainder = val - subtractor;

    cout << "Resto da divisao por 2 e " << remainder << endl;
}

// Resultado da divisão por 2
if(operation == 2) {
    int cont = 0;
    int result = val;
    while(result - 2 >= 0) {
        result = result - 2;
        cont = cont + 1;
    }
    cout << "Resultado da divisao por 2 e " << cont << endl;
}
return 0;
}

```

Código fonte em Assembly do MIPS:

```

# Disciplina: Arquitetura e Organização de Computadores
# Atividade: Avaliação 01 - Programação em Linguagem de Montagem
# Programa 01
# Grupo: - Gustavo Henrique Stahl Müller
#         - Paulo Henrique Rohling

.data
textEnter: .asciiz "Entre com um valor inteiro (maior do que zero):\n"
textEnterFail: .asciiz "Valor inválido.\n"
textOperationMenu: .asciiz "\nDigite uma operacao, sendo que:\n0 = encerra o programa;\n1 =
informa o resto da divisao por 2;\n2 = informa o resultado da divisao por 2.\n\n"
textOperationMenuFailed: .asciiz "Operação inválida.\n"
textOperation1: .asciiz "\nResto da divisão por 2 é: "
textOperation2: .asciiz "\nResultado da divisão por 2 é: "

.text

##### VALOR DO USUÁRIO
        # Mostra a mensagem de entrada de valor
        la $a0, textEnter                # Carrega a mensagem como argumento
        addi $v0, $zero, 4               # Carrega o comando de mostrar uma string
        syscall                          # Mostra a string

startBeginLoop:
        # Pega o valor do usuário, garantindo que ele é maior do que 0
        addi $v0, $zero, 5               # Carrega o comando de ler um inteiro
        syscall                          # Lê o inteiro
        add $s0, $v0, $zero              # Armazena o inteiro no registrador $s0
        bgt $s0, $zero, endBeginLoop     # Continua o programa se ele não for maior que
zero

        # Imprime a mensagem de erro na tela, caso o valor seja igual ou menor à 0
        la $a0, textEnterFail            # Carrega a mensagem de erro no argumento
        addi $v0, $zero, 4               # Carrega o comando de mostrar uma string
        syscall                          # Mostra a string

        # Volta para o início do loop
        j startBeginLoop

endBeginLoop:
#####
# MENU DE OPERAÇÕES
        # Mostra as operações disponíveis
        la $a0, textOperationMenu        # Carrega o texto das operações como argumento
        addi $v0, $zero, 4               # Carrega a operação de mostrar uma string
        syscall                          # Mostra a string

```

```

startOperationLoop:
    # Pega o valor do usuário, não permitindo entradas abaixo de 0 e maiores que 2
    addi $v0, $zero, 5          # Carrega o comando de ler um inteiro
    syscall                     # Lê o inteiro
    add $s1, $v0, $zero         # Armazena o inteiro no registrador $s1

    # Checa se ele é menor do que 0
    slti $t1, $s1, 0
    bne $t1, $zero, operationMenuFail

    # Checa se ele é maior do que 2
    bgt $s1, 2, operationMenuFail

    # Se passou pelos testes, vai para o fim da seleção de operações
    j endOperationLoop

operationMenuFail:
    # Imprime a mensagem de erro na tela, caso o valor seja menor do que 0 ou maior do que 2
    la $a0, textOperationMenuFailed # Carrega a mensagem de erro no argumento
    addi $v0, $zero, 4             # Carrega o comando de mostrar uma string
    syscall                         # Mostra a string

    # Volta para o início do loop
    j startOperationLoop

endOperationLoop:
    # Decide a operação que ele irá fazer
    beq $s1, 0, operation0        # Vai para a operação 0
    beq $s1, 1, operation1        # Vai para a operação 1
    beq $s1, 2, operation2        # Vai para a operação 2
##### OPERAÇÃO 0
operation0:
    # Termina o programa (operação 0)
    addi $v0, $zero, 10          # Carrega a operação de terminar o sistema
    syscall                      # Termina o sistema
##### OPERAÇÃO 1
operation1:
    # Faz o resto da divisão sem a instrução "mod"
    div $t0, $s0, 2              # t0 = floor(val/2)
    add $t0, $t0, $t0             # t0 = 2 * t0
    sub $t1, $s0, $t0            # t1 (resto) = s0 (valor) - t0 (resultado)

    # Mostra o texto inicial na tela
    la $a0, textOperation1       # Carrega o texto inicial do resultado como
argumento
    addi $v0, $zero, 4           # Carrega a operação de mostrar uma string
    syscall                      # Mostra a string

    # Concatena o resultado na tela
    add $a0, $t1, $zero           # Carrega o resto como argumento
    addi $v0, $zero, 1           # Carrega a operação de mostrar um inteiro
    syscall                      # Mostra o inteiro

    # Termina a execução
    j operation0
##### OPERAÇÃO 2
operation2:
    # Decresce o valor até chegar em 0
    addi $s1, $zero, 0           # Inicia o contador de vezes como 0
    add $t1, $s0, $zero          # t1 (cópia) = s0 (valor)

startOperation2Loop:
    subi $t2, $t1, 2             # t2 (temporário) = t0 (cópia) - 2
    blt $t2, 0, endOperation2Loop # Checa se ainda pode tirar 2, se não, para o loop

    # Como consegue tirar 2 e o resultado ainda vai ser maior que 0, continua tirando
    subi $t1, $t1, 2             # Retira 2 do resultado
    addi $s1, $s1, 1             # Adiciona um ao contador

    # Faz o loop novamente
    j startOperation2Loop

endOperation2Loop:
    # Mostra o texto inicial na tela
    la $a0, textOperation2       # Carrega o texto inicial do resultado como argumento
    addi $v0, $zero, 4           # Carrega a operação de mostrar uma string
    syscall                      # Mostra a string

```

```

# Concatena o resultado na tela
add $a0, $s1, $zero      # Carrega o contador como argumento
addi $v0, $zero, 1       # Carrega a operação de mostrar um inteiro
syscall                  # Mostra o inteiro

# Termina a execução
j operation0

#####

```

Capturas de Tela:

Pegando o valor:

```

Entre com um valor inteiro (maior do que zero):

```

Digitando o valor 9:

```

Entre com um valor inteiro (maior do que zero):
9

Digite uma operacao, sendo que:
0 = encerra o programa;
1 = informa o resto da divisao por 2;
2 = informa o resultado da divisao por 2.

```

Escolhendo a operação 0 (Sair do programa):

```

^
Digite uma operacao, sendo que:
0 = encerra o programa;
1 = informa o resto da divisao por 2;
2 = informa o resultado da divisao por 2.

0

-- program is finished running --

```

Escolhendo a operação 1 (Resto da Divisão):

```
Digite uma operacao, sendo que:
0 = encerra o programa;
1 = informa o resto da divisao por 2;
2 = informa o resultado da divisao por 2.

1

Resto da divisao por 2 é: 1
-- program is finished running --
```

Escolhendo a operação 2 (Resultado da Divisão):

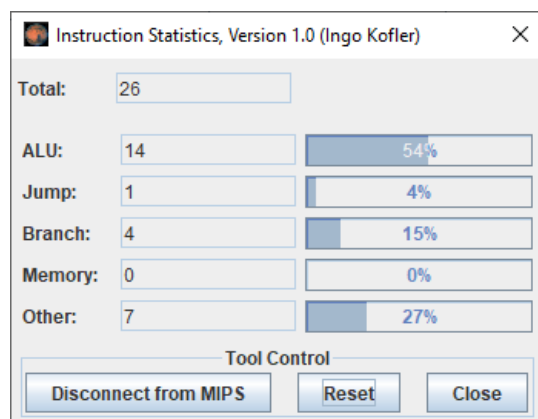
```
Digite uma operacao, sendo que:
0 = encerra o programa;
1 = informa o resto da divisao por 2;
2 = informa o resultado da divisao por 2.

2

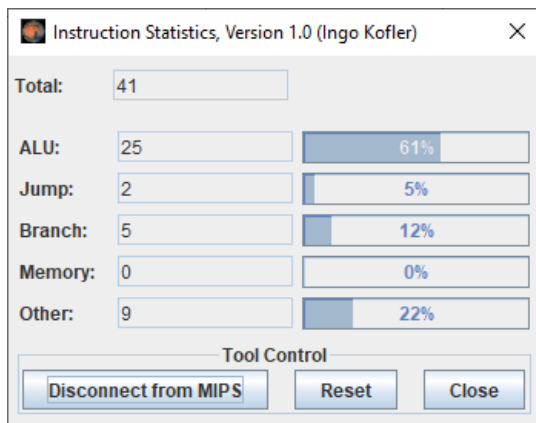
Resultado da divisao por 2 é: 4
-- program is finished running --
```

Instruction Statistics:

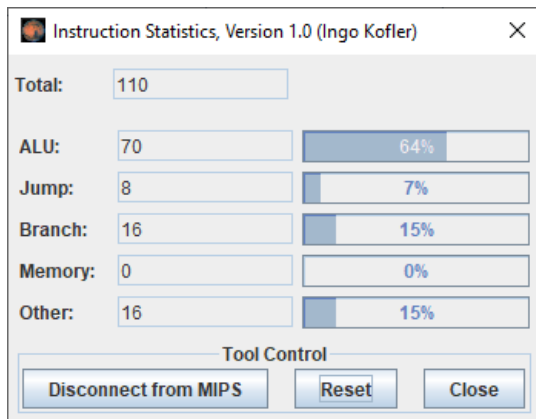
Realizando a operação 0 (terminar o programa):



Realizando a operação 1 (resto de divisão) com o valor “11”:



Realizando a operação 2 (divisão inteira) como valor “11”:



3. SEGUNDO PROGRAMA

“Implemente um programa que leia um vetor via console e, após a leitura do vetor, troque o conteúdo do vetor de ordem. Por fim, o programa deve imprimir esse novo vetor na tela”.

Código fonte em linguagem de alto nível (C++):

```
#include <iostream>

using namespace std;

int main() {
    int vetor[8] = {0,0,0,0,0,0,0,0};        // Inicialização do Vetor
    int tamanhoVetor;                        // Tamanho do Vetor
    int ultimaPosicao;                       // Armazena a Última posição, para facilitar o swap

    // Entrada do Tamanho do Vetor
    cout << "Entre com o tamanho do vetor (max. = 8): ";
    cin >> tamanhoVetor;

    // Continua pedindo um tamanho enquanto este não for válido
    while(tamanhoVetor < 1 || tamanhoVetor > 8) {
        cout << "Valor invalido" << endl;
        cout << "Entre com o tamanho do vetor (max. = 8): ";
        cin >> tamanhoVetor;
    }

    // Cálculo da última posição
    ultimaPosicao = tamanhoVetor - 1;

    // Solicita os valores para preencher o vetor
    cout << "\n\nSOLICITACAO DE VALORES DO VETOR: \n";
    for(int i = 0; i < tamanhoVetor; i++) {
        cout << "Vetor[" << i << "]: ";
        cin >> vetor[i];
    }

    // Faz o swap das posições
    for(int i = 0; i < tamanhoVetor/2; i++) {
        int temp = vetor[i];
        vetor[i] = vetor[ultimaPosicao - i];
        vetor[ultimaPosicao - i] = temp;
    }

    // Imprime o vetor invertido
    cout << "\n\nIMPRESSAO DO VETOR INVERTIDO:\n";
    for(int i = 0; i < tamanhoVetor; i++) {
        cout << "\nVetor[" << i << "]: " << vetor[i];
    }

    cout << "\n\n";
    return 0;
}
```


Código fonte em Assembly do MIPS:

```
# Disciplina: Arquitetura e Organização de Computadores
# Atividade: Avaliação 01 - Programação em Linguagem de Montagem
# Programa 02
# Grupo: - Gustavo Henrique Stahl Müller
#        - Paulo Henrique Rohling

.data
    Vetor:                .word    0, 0, 0, 0, 0, 0, 0, 0    # Vetor principal

    # Textos que serão impressos:
    inputText:    .asciiz  "Entre com o tamanho do vetor (máx. = 8): "
    errorText:    .asciiz  "Valor inválido.\n"
    fillVector:   .asciiz  "\n\nSolicitação de Valores do Vetor: \n"
    printVector:  .asciiz  "\n\nImpressão do Vetor Invertido:\n"
    beginString:  .asciiz  "Vetor["
    beginString2: .asciiz  "\nVetor["
    endString:    .asciiz  "]: "

.text

    # Inicializa $t5 com o valor máximo do vetor
    addi $t5, $zero, 8

RequestValue:
    # Impressão do inputText
    addi $v0, $zero, 4
    la $a0, inputText
    syscall

    # Inserção do usuário em $t0(tamanhoVetor)
    addi $v0, $zero, 5
    syscall
    addi $t0, $v0, 0

    # Passa para a checagem
    j Check1

Check1:
    slti $t7, $t0, 1        # Se o valor inserido é menor do que 1
    beq $t7, $zero, Check2  # Se a condição anterior resultou em falso, desvia para Check2
    j WhileInvalid

Check2:
    bgt $t0, $t5, WhileInvalid # Se o valor inserido é maior que 8, desvia para WhileInvalid
    j SetLastPosition

WhileInvalid:
    # Impressão do errorText
    addi $v0, $zero, 4
    la $a0, errorText
    syscall
    j RequestValue

SetLastPosition:
    subi $t1, $t0, 1
    j GetValues

GetValues:
    # Impressão do fillVector
    addi $v0, $zero, 4
    la $a0, fillVector
    syscall

    # Armazena em $t2 o endereço base do vetor
    la $t2, Vetor

    addi $t7, $zero, 0        # Inicializa o contador com 0
    addi $t6, $t0, 0        # Define o limite do laço FOR como o tamanho do Vetor
```

```

For:
    slt $t8, $t7, $t6          # Se o valor do contador é menor do que o tamanho do Vetor
    beq $t8, $zero, Swap      # Se a condição anterior resultou em verdadeiro, desvia para Swap

    # Impressão da beginString = "Vetor["
    addi $v0, $zero, 4
    la $a0, beginString
    syscall

    # Impressão da posição atual do vetor
    addi $v0, $zero, 1
    addi $a0, $t7, 0
    syscall

    # Impressão da endString = "]: "
    addi $v0, $zero, 4
    la $a0, endString
    syscall

    # Achando endereço da posição atual do vetor
    add $t9, $t7, $t7          # 2 * contador
    add $t9, $t9, $t9          # 4 * contador
    add $t9, $t9, $t2          # Soma o deslocamento de posição com o endereço do início do vetor

    # Inserção do usuário na posição atual do vetor
    addi $v0, $zero, 5
    syscall
    sw $v0, 0($t9)

    # Incrementa o Contador e desvia para o início
    addi $t7, $t7, 1
    j For

Swap:
    # Divide tamanhoVetor($t0) por 2, ou seja, desloca-o para a direita
    srl $s0, $t0, 1

    addi $t7, $zero, 0          # Inicializa o contador com 0
    addi $t6, $s0, 0           # Define o limite do laço FOR como o tamanho do Vetor

ForSwap:
    slt $t8, $t7, $t6          # Se o valor do contador é menor do que o tamanho do Vetor
    beq $t8, $zero, PrintVector # Se a condição anterior resultou em verdadeiro, desvia para PrintVector

    # Achando endereço da posição atual do vetor
    add $t9, $t7, $t7          # 2 * contador
    add $t9, $t9, $t9          # 4 * contador
    add $t9, $t9, $t2          # Soma o deslocamento de posição com o endereço do início do vetor

    # Salvando o valor do endereço atual do vetor em um registrador
    lw $s1, 0($t9)             # temp = vetor[i]

    # Calculando (ultimaPosicao - Valor do contador)
    sub $s2, $t1, $t7          # ultimaPosicao - i

    # Achando endereço da posição (ultimaPosicao - i) do vetor
    add $s3, $s2, $s2          # 2 * contador
    add $s3, $s3, $s3          # 4 * contador
    add $s3, $s3, $t2          # Soma o deslocamento de posição com o endereço do início do vetor

    # Salvando o valor da posição (ultimaPosicao - i) do vetor em um registrador
    lw $s4, 0($s3)             # vetor[ultimaPosicao - i]

    # Fazendo a atribuição: vetor[i] = vetor[ultimaPosicao - i]
    sw $s4, 0($t9)

    # Fazendo a atribuição: vetor[ultimaPosicao - i] = temp
    sw $s1, 0($s3)

    # Incrementa o Contador e desvia para o início
    addi $t7, $t7, 1
    j ForSwap

```

```

PrintVector:
    # Impressão do printVector
    addi $v0, $zero, 4
    la $a0, printVector
    syscall

    addi $t7, $zero, 0          # Inicializa o contador com 0
    addi $t6, $t0, 0           # Define o limite do laço FOR como o tamanho do Vetor

ForPrint:
    slt $t8, $t7, $t6          # Se o valor do contador é menor do que o tamanho do Vetor
    beq $t8, $zero, Finish     # Se a condição anterior resultou em verdadeiro, desvia para PrintVector

    # Achando endereço da posição atual do vetor
    add $t9, $t7, $t6          # 2 * contador
    add $t9, $t9, $t9          # 4 * contador
    add $t9, $t9, $t2          # Soma o deslocamento de posição com o endereço do início do vetor

    # Salvando o valor do endereço atual do vetor em um registrador
    lw $s1, 0($t9)             # vetor[i]

    # Impressão da beginString = "\nVetor["
    addi $v0, $zero, 4
    la $a0, beginString2
    syscall

    # Impressão da posição atual do vetor
    addi $v0, $zero, 1
    addi $a0, $t7, 0
    syscall

    # Impressão da endString2 = "]: "
    addi $v0, $zero, 4
    la $a0, endString
    syscall

    # Impressão do valor da posição atual do vetor
    addi $v0, $zero, 1
    addi $a0, $s1, 0
    syscall

    addi $t7, $t7, 1
    j ForPrint

Finish:
    nop

```

Capturas de Tela:

Requisitando o tamanho do vetor:

```
Entre com o tamanho do vetor (máx. = 8): |
```

Solicita ao usuário a quantidade de valores que ele estabeleceu:

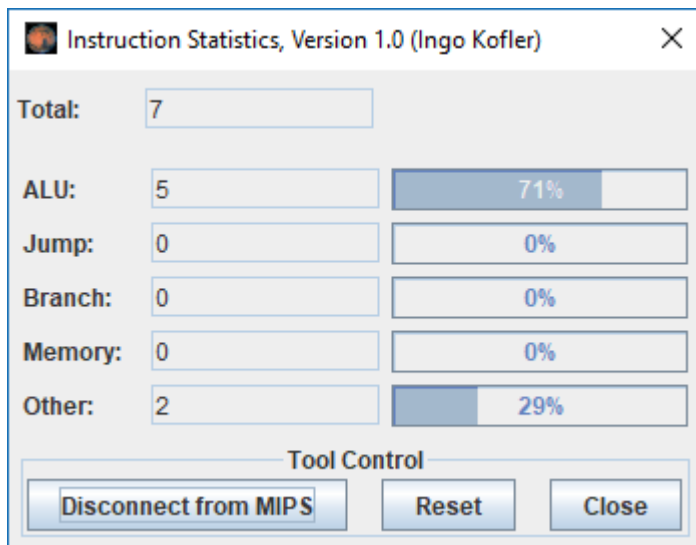
```
Solicitação de Valores do Vetor:  
Vetor[0]: 1  
Vetor[1]: 2  
Vetor[2]: 3  
Vetor[3]: 4  
Vetor[4]: 5  
Vetor[5]: 6  
Vetor[6]: 7  
Vetor[7]: 8
```

Impressão do Vetor invertido:

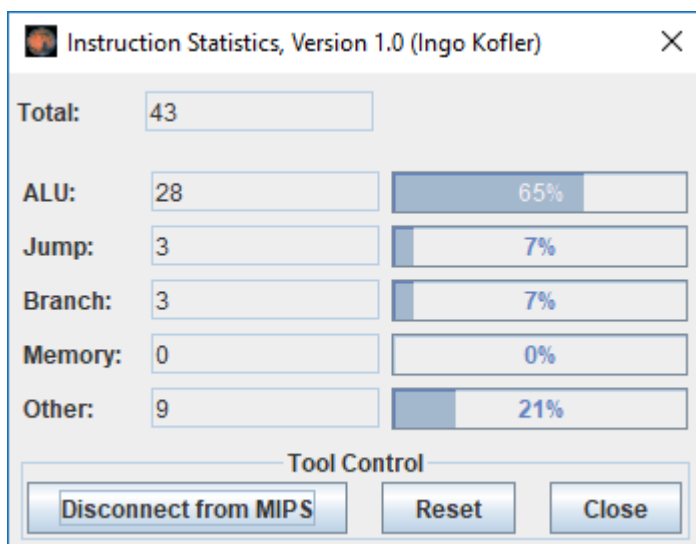
```
Impressão do Vetor Invertido:  
  
Vetor[0]: 8  
Vetor[1]: 7  
Vetor[2]: 6  
Vetor[3]: 5  
Vetor[4]: 4  
Vetor[5]: 3  
Vetor[6]: 2  
Vetor[7]: 1  
-- program is finished running (dropped off bottom) --
```

Instruction Statistics:

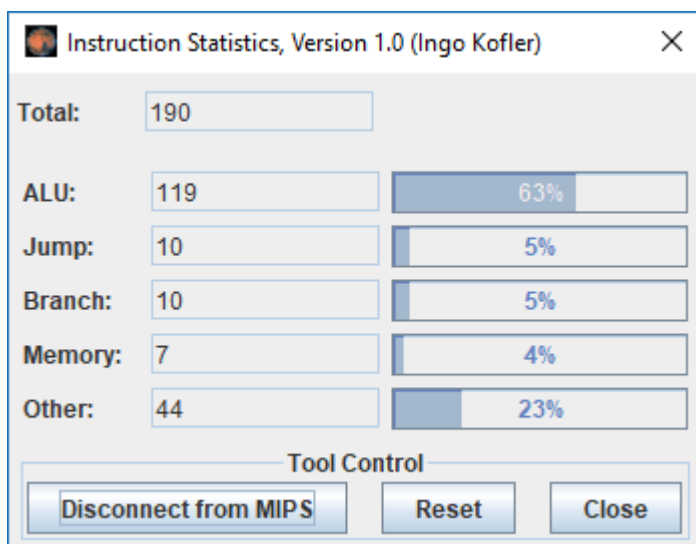
Inserindo o tamanho do Vetor:



Antes da inserção de todos os valores do vetor:



Após a inserção de quase todos os valores do vetor:



Após o término da execução do programa:

